

# TaskOrbit Conversational Agent

## Build & Deploy Documentation

### Sprint 13 — Final Release Consolidated Documentation

Generated: 2026-07-12

Source: GitHub Wiki ( [amos2026ss04-taskorbit-conversational-agent.wiki](#) )

Built by: AMOS SS2026 Team 04, in collaboration with Meisterwerk App GmbH, Berlin

---

## Table of Contents

1. [Getting Started](#)
  2. [Build and Deploy Documentation](#)
  3. [CI/CD Configuration](#)
  4. [Cloud Deployment](#)
  5. [Database Setup](#)
  6. [LiveKit Cloud Setup](#)
  7. [LLM Integration](#)
  8. [Self-Hosted Inference](#)
  9. [SBOM & Legal Notices](#)
-

# Getting Started

# Getting Started

Two ways to run TaskOrbit locally — pick one:

| Path                 | What you need             | Best for                          |
|----------------------|---------------------------|-----------------------------------|
| Docker (recommended) | Docker only               | Running the full stack quickly    |
| Local dev            | Python + Poetry + Node.js | Active development, running tests |

## 1. Install Docker

The only hard requirement for the Docker path.

### Mac

```
# Install Docker Desktop from https://docs.docker.com/desktop/install/mac-install/
## Then verify:
docker --version
docker compose version
```

### Windows

```
# Install Docker Desktop from https://docs.docker.com/desktop/install/windows-install/
## Enable WSL2 backend when prompted (recommended).
## Then verify in PowerShell or WSL2 terminal:
docker --version
docker compose version
```

### Linux (Ubuntu/Debian)

```
sudo apt update
sudo apt install -y docker.io docker-compose-plugin
sudo usermod -aG docker $USER # allows running docker without sudo
newgrp docker # apply group change without logout
docker --version
docker compose version
```

## 2. Clone the repo

### Mac / Linux / Windows WSL2

```
git clone https://github.com/amosproj/amos2026ss04-taskorbit-conversational-agent.git
cd amos2026ss04-taskorbit-conversational-agent
```

### Windows (PowerShell)

```
git clone https://github.com/amosproj/amos2026ss04-taskorbit-conversational-agent.git
cd amos2026ss04-taskorbit-conversational-agent
```

### 3. Start the full stack (Docker — no other installs needed)

Python, Poetry, Node.js, and PostgreSQL are all handled inside the containers automatically.

#### Mac / Linux / Windows WSL2

```
docker compose up --build
```

#### Windows (PowerShell)

```
docker compose up --build
```

This starts three containers:

- **PostgreSQL** on port 5432
- **Backend** (FastAPI) on port 8000
- **Frontend** (Vite) on port 5173

*First run takes a few minutes while Docker pulls base images and installs dependencies. Subsequent runs are much faster.*

---

### 4. Run the backend locally (outside Docker)

Only needed for active backend development.

#### Install Python 3.11

##### Mac

```
brew install python@3.11
python3.11 --version
```

##### Windows

```
# Download from https://www.python.org/downloads/
## During install – check "Add Python to PATH"
python --version
```

##### Linux (Ubuntu/Debian)

```
sudo apt update
sudo apt install -y python3.11 python3.11-venv
python3.11 --version
```

#### Install Poetry

##### Mac / Linux

```
curl -sSL https://install.python-poetry.org | python3 -
export PATH="$HOME/.local/bin:$PATH" # add to ~/.bashrc or ~/.zshrc to persist
poetry --version
```

##### Windows (PowerShell)

```
(Invoke-WebRequest -Uri https://install.python-poetry.org -UseBasicParsing).Content | python -
## Then add Poetry to PATH via System Properties → Environment Variables
## Default location: %APPDATA%\Python\Scripts
poetry --version
```

## Start the backend

### Mac / Linux / Windows WSL2

```
cd backend
poetry install
poetry run uvicorn taskorbit.api.main:app --reload --port 8000
```

### Windows (PowerShell)

```
cd backend
poetry install
poetry run uvicorn taskorbit.api.main:app --reload --port 8000
```

Check it's alive:

### Mac / Linux / Windows WSL2

```
curl http://localhost:8000/health
```

### Windows (PowerShell)

```
Invoke-WebRequest http://localhost:8000/health
```

---

## 5. Run the test suite

### Mac / Linux / Windows WSL2

```
cd backend
poetry run pytest tests/ -v
```

### Windows (PowerShell)

```
cd backend
poetry run pytest tests/ -v
```

Expected output — all tests passing:

```
tests/test_task_loader.py::test_load_valid_task          PASSED
tests/test_task_loader.py::test_load_missing_file        PASSED
tests/test_task_loader.py::test_load_invalid_json        PASSED
tests/test_engine.py::test_first_turn_returns_first_message PASSED
tests/test_engine.py::test_session_marked_complete       PASSED
... (16 total)
```

---

## 6. Try the pipeline endpoint

The sprint 1 pipeline runs without API keys — it uses a stubbed LLM response.

**First call — starts a new session, returns the agent greeting:**

### Mac / Linux / Windows WSL2

```
curl -X POST http://localhost:8000/pipeline/run \
  -H "Content-Type: application/json" \
  -d '{
```

```
"task_path": "tasks/book_appointment.json"
}'
```

### Windows (PowerShell)

```
Invoke-RestMethod -Method POST http://localhost:8000/pipeline/run `
-ContentType "application/json" `
-Body '{"task_path": "tasks/book_appointment.json"}'
```

Response:

```
{
  "session_id": "a3f2c1d0-...",
  "reply": "Hi, thanks for calling Mueller Plumbing. How can I help you today?",
  "current_step": 1,
  "is_complete": false
}
```

### Subsequent calls — continue the conversation:

### Mac / Linux / Windows WSL2

```
curl -X POST http://localhost:8000/pipeline/run \
-H "Content-Type: application/json" \
-d '{
  "task_path": "tasks/book_appointment.json",
  "session_id": "a3f2c1d0-...",
  "message": "My name is Ana and I need to book a plumbing appointment"
}'
```

### Windows (PowerShell)

```
Invoke-RestMethod -Method POST http://localhost:8000/pipeline/run `
-ContentType "application/json" `
-Body '{"task_path": "tasks/book_appointment.json", "session_id": "a3f2c1d0-...", "message": "My name is Ana and I need to book a plumbing appointment"}'
```

---

## 7. Run the frontend locally (outside Docker)

Only needed for active frontend development.

### Install Node.js 20+

#### Mac

```
brew install node@20
node --version
```

#### Windows

```
# Download from https://nodejs.org/en/download
## LTS version recommended. Installer adds Node to PATH automatically.
node --version
```

#### Linux (Ubuntu/Debian)

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs
node --version
```

## Start the frontend

### Mac / Linux / Windows WSL2

```
cd frontend
npm install
npm run dev
```

### Windows (PowerShell)

```
cd frontend
npm install
npm run dev
```

Frontend available at `http://localhost:5173` .

---

## Environment variables

Create a `backend/.env` file. Docker Compose picks this up automatically.

### Mac / Linux / Windows WSL2

```
cp backend/.env.example backend/.env
## then edit backend/.env with your values
```

### Windows (PowerShell)

```
Copy-Item backend\.env.example backend\.env
## then edit backend\.env with your values
```

```
# Database
DATABASE_URL=postgres://taskorbit:taskorbit@localhost:5432/taskorbit
```

## Sprint 2 – add these when API keys are available

**OPENAI\_API\_KEY=sk-...**

**DEEPGRAM\_API\_KEY=...**

**ELEVENLABS\_API\_KEY=...**

**LIVEKIT\_URL=...**

**LIVEKIT\_API\_KEY=...**

```
LIVEKIT_API_SECRET=...
```

⚠ **Never commit API keys to the repo.** The `.env` file is in `.gitignore`.

## Common issues

### Port 5432 already in use ( `docker compose up` fails)

Mac:

```
brew services stop postgresql
```

Linux:

```
sudo service postgresql stop
```

Windows (PowerShell as Administrator):

```
net stop postgresql
```

### poetry: command not found (Mac/Linux)

```
export PATH="$HOME/.local/bin:$PATH"
```

Add this line to `~/.bashrc` or `~/.zshrc` to persist across sessions.

### poetry not recognised (Windows)

Add `%APPDATA%\Python\Scripts` to your PATH via System Properties → Environment Variables → User variables → Path → Edit.

### /pipeline/run returns 404 for task file

The `task_path` is relative to where the backend process runs. Use `tasks/book_appointment.json` and confirm the file exists under `backend/tasks/`.

See also: [API Reference](#) · [Agent Task Example](#) · [LiveKit Cloud Setup](#)

## # Getting Started

Two ways to run TaskOrbit locally — pick one:

| Path                        | What you need             | Best for                          |
|-----------------------------|---------------------------|-----------------------------------|
| <b>Docker</b> (recommended) | Docker only               | Running the full stack quickly    |
| <b>Local dev</b>            | Python + Poetry + Node.js | Active development, running tests |

## 1. Install Docker

The only hard requirement for the Docker path.

### Mac

```
# Install Docker Desktop from https://docs.docker.com/desktop/install/mac-install/
# Then verify:
```

```
docker --version
docker compose version
```

## Windows

```
# Install Docker Desktop from https://docs.docker.com/desktop/install/windows-install/
# Enable WSL2 backend when prompted (recommended).
# Then verify in PowerShell or WSL2 terminal:
docker --version
docker compose version
```

## Linux (Ubuntu/Debian)

```
sudo apt update
sudo apt install -y docker.io docker-compose-plugin
sudo usermod -aG docker $USER # allows running docker without sudo
newgrp docker                 # apply group change without logout
docker --version
docker compose version
```

---

## 2. Clone the repo

### Mac / Linux / Windows WSL2

```
git clone https://github.com/amosproj/amos2026ss04-taskorbit-conversational-agent.git
cd amos2026ss04-taskorbit-conversational-agent
```

### Windows (PowerShell)

```
git clone https://github.com/amosproj/amos2026ss04-taskorbit-conversational-agent.git
cd amos2026ss04-taskorbit-conversational-agent
```

---

## 3. Start the full stack (Docker — no other installs needed)

Python, Poetry, Node.js, and PostgreSQL are all handled inside the containers automatically.

### Mac / Linux / Windows WSL2

```
docker compose up --build
```

### Windows (PowerShell)

```
docker compose up --build
```

This starts three containers:

- **PostgreSQL** on port 5432
- **Backend** (FastAPI) on port 8000
- **Frontend** (Vite) on port 5173

*First run takes a few minutes while Docker pulls base images and installs dependencies. Subsequent runs are much faster.*

---



## 4. Run the backend locally (outside Docker)

Only needed for active backend development.

### Install Python 3.11

#### Mac

```
brew install python@3.11
python3.11 --version
```

#### Windows

```
# Download from https://www.python.org/downloads/
# During install – check "Add Python to PATH"
python --version
```

#### Linux (Ubuntu/Debian)

```
sudo apt update
sudo apt install -y python3.11 python3.11-venv
python3.11 --version
```

### Install Poetry

#### Mac / Linux

```
curl -sSL https://install.python-poetry.org | python3 -
export PATH="$HOME/.local/bin:$PATH" # add to ~/.bashrc or ~/.zshrc to persist
poetry --version
```

#### Windows (PowerShell)

```
(Invoke-WebRequest -Uri https://install.python-poetry.org -UseBasicParsing).Content | python -
# Then add Poetry to PATH via System Properties → Environment Variables
# Default location: %APPDATA%\Python\Scripts
poetry --version
```

### Start the backend

#### Mac / Linux / Windows WSL2

```
cd backend
poetry install
poetry run uvicorn taskorbit.api.main:app --reload --port 8000
```

#### Windows (PowerShell)

```
cd backend
poetry install
poetry run uvicorn taskorbit.api.main:app --reload --port 8000
```

Check it's alive:

## Mac / Linux / Windows WSL2

```
curl http://localhost:8000/health
```

## Windows (PowerShell)

```
Invoke-WebRequest http://localhost:8000/health
```

---

## 5. Run the test suite

### Mac / Linux / Windows WSL2

```
cd backend
poetry run pytest tests/ -v
```

### Windows (PowerShell)

```
cd backend
poetry run pytest tests/ -v
```

Expected output — all tests passing:

```
tests/test_task_loader.py::test_load_valid_task          PASSED
tests/test_task_loader.py::test_load_missing_file        PASSED
tests/test_task_loader.py::test_load_invalid_json        PASSED
tests/test_engine.py::test_first_turn_returns_first_message PASSED
tests/test_engine.py::test_session_marked_complete      PASSED
... (16 total)
```

---

## 6. Try the pipeline endpoint

The sprint 1 pipeline runs without API keys — it uses a stubbed LLM response.

**First call — starts a new session, returns the agent greeting:**

### Mac / Linux / Windows WSL2

```
curl -X POST http://localhost:8000/pipeline/run \
  -H "Content-Type: application/json" \
  -d '{
    "task_path": "tasks/book_appointment.json"
  }'
```

### Windows (PowerShell)

```
Invoke-RestMethod -Method POST http://localhost:8000/pipeline/run `
  -ContentType "application/json" `
  -Body '{"task_path": "tasks/book_appointment.json"}'
```

Response:

```
{
  "session_id": "a3f2c1d0-...",
```

```
"reply": "Hi, thanks for calling Mueller Plumbing. How can I help you today?",
"current_step": 1,
"is_complete": false
}
```

**Subsequent calls — continue the conversation:**

**Mac / Linux / Windows WSL2**

```
curl -X POST http://localhost:8000/pipeline/run \
-H "Content-Type: application/json" \
-d '{
  "task_path": "tasks/book_appointment.json",
  "session_id": "a3f2c1d0-...",
  "message": "My name is Ana and I need to book a plumbing appointment"
}'
```

**Windows (PowerShell)**

```
Invoke-RestMethod -Method POST http://localhost:8000/pipeline/run `
-ContentType "application/json" `
-Body '{"task_path": "tasks/book_appointment.json", "session_id": "a3f2c1d0-...", "message": "My name is
Ana and I need to book a plumbing appointment"}'
```

---

## 7. Run the frontend locally (outside Docker)

Only needed for active frontend development.

**Install Node.js 20+**

**Mac**

```
brew install node@20
node --version
```

**Windows**

```
# Download from https://nodejs.org/en/download
# LTS version recommended. Installer adds Node to PATH automatically.
node --version
```

**Linux (Ubuntu/Debian)**

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs
node --version
```

**Start the frontend**

**Mac / Linux / Windows WSL2**

```
cd frontend
npm install
```

```
npm run dev
```

### Windows (PowerShell)

```
cd frontend
npm install
npm run dev
```

Frontend available at `http://localhost:5173` .

---

## Environment variables

Create a `backend/.env` file. Docker Compose picks this up automatically.

### Mac / Linux / Windows WSL2

```
cp backend/.env.example backend/.env
# then edit backend/.env with your values
```

### Windows (PowerShell)

```
Copy-Item backend\.env.example backend\.env
# then edit backend\.env with your values
```

```
# Database
DATABASE_URL=postgresql://taskorbit:taskorbit@localhost:5432/taskorbit

# Sprint 2 – add these when API keys are available
# OPENAI_API_KEY=sk-...
# DEEPGRAM_API_KEY=...
# ELEVENLABS_API_KEY=...
# LIVEKIT_URL=...
# LIVEKIT_API_KEY=...
# LIVEKIT_API_SECRET=...
```

 **Never commit API keys to the repo.** The `.env` file is in `.gitignore` .

---

## Common issues

### Port 5432 already in use ( `docker compose up` fails)

Mac:

```
brew services stop postgresql
```

Linux:

```
sudo service postgresql stop
```

Windows (PowerShell as Administrator):

```
net stop postgresql
```

**poetry: command not found (Mac/Linux)**

```
export PATH="$HOME/.local/bin:$PATH"
```

Add this line to `~/.bashrc` or `~/.zshrc` to persist across sessions.

**poetry not recognised (Windows)**

Add `%APPDATA%\Python\Scripts` to your PATH via System Properties → Environment Variables → User variables → Path → Edit.

**/pipeline/run returns 404 for task file**

The `task_path` is relative to where the backend process runs. Use `tasks/book_appointment.json` and confirm the file exists under `backend/tasks/`.

---

See also: [API Reference](#) · [Agent Task Example](#) · [LiveKit Cloud Setup](#)

---

# Build and Deploy Documentation

## TaskOrbit Build & Deploy Documentation

Issue: [#74 Documentation](#)

This page is the developer-facing quick start for running TaskOrbit locally. For deep dives, see:

- [CI/CD Configuration](#) — all 7 GitHub Actions workflows in detail
  - [Cloud Deployment](#) — how the `prod` deployment, GCP auth, and Terraform infrastructure work
  - [LiveKit Cloud Setup](#) — full voice/LiveKit credential walkthrough
  - [System Architecture](#) — pipeline and backend structure
- 

### Prerequisites

#### Docker path (recommended):

- Docker Desktop — handles Python, Node.js, and PostgreSQL automatically. No other runtimes required.
- Git

#### Local development path (outside Docker):

- Python 3.11
- Poetry — Python dependency and virtualenv manager
- Node.js 20 or later with npm
- Git

### Local Development

#### Running the Full Stack with Docker

##### Step 1. Clone the repository.

```
git clone https://github.com/amosproj/amos2026ss04-taskorbit-conversational-agent.git
cd amos2026ss04-taskorbit-conversational-agent
```

##### Step 2. Copy environment files.

```
cp backend/.env.example backend/.env
cp frontend/.env.example frontend/.env.local
```

Voice requires LiveKit Cloud credentials in `backend/.env` — see [LiveKit Cloud Setup](#) for how to get them. Without them, the text path still works.

##### Step 3. Start the stack.

```
docker compose up --build
```

`docker-compose.yml` defines **8 services** with no profile gating, so this single command starts all of them: `postgres` (host port **5435**, mapped to container `5432` — deliberately offset to avoid clashing with a local Postgres install), `backend` (FastAPI, `:8000`), `worker` (LiveKit voice agent, `:8001`, exposes `/metrics`), `frontend` (Vite, `:5173`), `prometheus` (`:9090`), `loki` (`:3100`), `promtail`, and `grafana` (`:3000`, anonymous admin login — **dev only**, never expose this configuration outside your machine).

##### Step 4. Verify the backend.

```
curl http://localhost:8000/health
```

Expected response: `{"status": "ok", "service": "taskorbit-backend", "version": "0.1.0"}`

**Step 5.** Open <http://localhost:5173> to access the frontend. Grafana is at <http://localhost:3000> if you want to look at metrics/logs locally.

To stop: `docker compose down`

### Running the Backend Locally (Outside Docker)

```
docker compose up -d postgres
cd backend
poetry install
DATABASE_URL="postgresql://taskorbit:taskorbit@localhost:5435/taskorbit" \
  poetry run taskorbit-api --reload --port 8000
```

The `DATABASE_URL` override is required here: the default in `config.py` points at port `5432` (correct for container-to-container traffic inside `docker-compose`), but the host-exposed port is `5435`. `taskorbit-api` is the same entrypoint used in CI (see [CI/CD Configuration](#)).

### Running the Frontend Locally (Outside Docker)

```
cd frontend
npm install
npm run dev
```

Frontend available at <http://localhost:5173>.

### Environment Variables

All credentials go in `backend/.env`. Never commit this file — it is gitignored.

| Variable                                                                                           | Purpose                                                                                                                                                                                             |
|----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>DATABASE_URL</code>                                                                          | PostgreSQL connection string. Default: <code>postgresql://taskorbit:taskorbit@localhost:5432/taskorbit</code> — override to port <code>5435</code> when connecting from outside Docker (see above). |
| <code>OPENAI_API_KEY</code> / <code>OPENAI_MODEL</code>                                            | OpenAI provider. Default model: <code>gpt-4o-mini</code>                                                                                                                                            |
| <code>GOOGLE_API_KEY</code> / <code>GOOGLE_MODEL</code>                                            | Google Gemini provider. Default model: <code>gemini-2.5-flash</code>                                                                                                                                |
| <code>OPENROUTER_API_KEY</code> / <code>OPENROUTER_MODEL</code>                                    | OpenRouter provider (open-source models, free tier available)                                                                                                                                       |
| <code>OLLAMA_BASE_URL</code> / <code>OLLAMA_MODEL</code>                                           | Self-hosted Ollama provider. Empty <code>OLLAMA_BASE_URL</code> disables it.                                                                                                                        |
| <code>DEEPGRAM_API_KEY</code> / <code>DEEPGRAM_MODEL</code> / <code>DEEPGRAM_LANGUAGE</code>       | Deepgram — can serve STT or TTS, per agent                                                                                                                                                          |
| <code>ELEVENLABS_API_KEY</code> / <code>ELEVENLABS_VOICE_ID</code> / <code>ELEVENLABS_MODEL</code> | ElevenLabs — can serve STT or TTS, per agent                                                                                                                                                        |
| <code>LIVEKIT_URL</code> / <code>LIVEKIT_API_KEY</code> / <code>LIVEKIT_API_SECRET</code>          | LiveKit Cloud credentials — see <a href="#">LiveKit Cloud Setup</a>                                                                                                                                 |

TaskOrbit supports **four** LLM providers (OpenAI, Google/Gemini, OpenRouter, Ollama) selected per-agent, and **two** dual-capable STT/TTS providers (Deepgram, ElevenLabs) also selected per-agent — see [System Architecture §3–4](#) for how provider selection works in code.

For the frontend, set `VITE_LIVEKIT_URL` in `frontend/.env.local` to the same LiveKit server URL.

## Database

TaskOrbit uses PostgreSQL 16 managed via SQLAlchemy (async) and Alembic.

Start the database:

```
docker compose up -d postgres
```

Run all pending migrations:

```
cd backend
poetry run alembic upgrade head
```

To generate a new migration after changing SQLAlchemy models:

```
cd backend
poetry run alembic revision --autogenerate -m "describe the change"
```

Review the generated file in `backend/migrations/versions/` before applying it.

## Testing

### Backend Tests

```
cd backend
poetry run pytest tests/ -v
```

All tests use mocks — no running database or external API keys required.

```
poetry run ruff check .
poetry run ruff format --check .
```

### Frontend Tests

```
cd frontend
npm run lint
npm run format:check
```

### Pre-commit Hooks

Install:

```
poetry -C backend run pre-commit install
cd frontend && npm install
```

Every commit automatically runs trailing whitespace checks, ruff lint and format for Python, and ESLint and Prettier for TypeScript. If a hook auto-fixes a file, the commit aborts — re-stage and re-commit. Full detail in [CI/CD Configuration](#).



## CI/CD Pipeline

TaskOrbit runs **7** GitHub Actions workflows, not all of which fire on every push. Full detail — triggers, jobs, troubleshooting — is in [CI/CD Configuration](#).

| Workflow                   | Trigger                                              | What it checks                                                       |
|----------------------------|------------------------------------------------------|----------------------------------------------------------------------|
| backend-lint.yml           | Every push/PR, all branches                          | ruff check, ruff format --check, pytest                              |
| frontend-lint.yml          | Every push/PR, all branches                          | ESLint, Prettier --check                                             |
| start-web-service.yml      | Every push, all branches                             | Boots FastAPI + curls /health, npm run build                         |
| validate-agent-configs.yml | PR opened/synchronize/reopened                       | scripts/validate_agent_configs.py                                    |
| verify-sbom.yml            | Push to main/prod, PRs, manual                       | SBOM + legal-notices freshness (see below)                           |
| benchmark.yml              | Push to feature/benchmarking-environment, manual     | Dry-run of the benchmarking harness (#86)                            |
| deploy.yml                 | After start-web-service.yml succeeds on prod, manual | Builds & deploys to Cloud Run — see <a href="#">Cloud Deployment</a> |

Branch protection on `main` requires `backend-lint.yml`, `frontend-lint.yml`, and `start-web-service.yml` to pass before a pull request can be merged.

### SBOM generation

`verify-sbom.yml` installs [Syft](#) and runs `./scripts/generate-sbom.sh` to regenerate `Documentation/sbom.cyclonedx.json` and `frontend/src/generated/legal-notices.json`, then fails the build if the committed artifacts don't match what the script produces. The SBOM is scripted and CI-enforced, not hand-maintained — see [SBOM & Legal Notices](#) for the full policy.

## Cloud Deployment

TaskOrbit deploys to Google Cloud Platform (region `europe-west3`) via Terraform and a GitHub Actions workflow ( `deploy.yml` ). Full detail — authentication, secrets, infrastructure, manual break-glass deploys — is in [Cloud Deployment](#). Summary:

- `deploy.yml` triggers when `start-web-service.yml` completes successfully on the `prod` branch (not `main`), or via manual `workflow_dispatch`.
  - It builds and pushes 3 images ( `backend`, `frontend`, `grafana` ) to Artifact Registry, then deploys 4 Cloud Run services ( `taskorbit-backend`, `taskorbit-worker` — reusing the backend image, `taskorbit-frontend`, `taskorbit-grafana` ).
  - GCP authentication currently uses a static service-account key ( `GCP_SA_KEY` secret). Workload Identity Federation is already provisioned in `terraform/modules/iam` but **not yet wired into `deploy.yml`** — this is a known, tracked migration, not the current behavior.
  - Never push directly to `prod`. Merge feature branches to `main`, verify, then open a merge request from `main` to `prod`.
-

# CI/CD Configuration

## CI / CD — TaskOrbit Conversational Agent

Issue: [#74 Documentation](#)

This document is the team-facing guide to the GitHub Actions pipeline. It covers what each workflow does, how to reproduce its checks locally, how to debug a red build, and how to extend the pipeline. There are **7** workflow files, not 3 — an earlier draft of this page only documented the two lint workflows and the smoke test. See [Cloud Deployment](#) for the full deployment/infrastructure story behind `deploy.yml` .

### Where the workflow files live

| File                                                      | Badge label                   | What it does                                                                                       |
|-----------------------------------------------------------|-------------------------------|----------------------------------------------------------------------------------------------------|
| <code>.github/workflows/backend-lint.yml</code>           | <b>Backend Linting</b>        | install → ruff check → ruff format --check → pytest                                                |
| <code>.github/workflows/frontend-lint.yml</code>          | <b>Frontend Linting</b>       | npm ci → ESLint → Prettier --check                                                                 |
| <code>.github/workflows/start-web-service.yml</code>      | <b>Start Web Service</b>      | Smoke test — boot FastAPI + curl /health; build the frontend bundle                                |
| <code>.github/workflows/validate-agent-configs.yml</code> | <b>Validate agent configs</b> | Runs <code>scripts/validate_agent_configs.py</code> against PR changes                             |
| <code>.github/workflows/verify-sbom.yml</code>            | <b>Verify SBOM</b>            | Regenerates the CycloneDX SBOM + frontend legal notices, fails if committed artifacts are stale    |
| <code>.github/workflows/benchmark.yml</code>              | <b>Benchmark Environment</b>  | Issue #86 — builds the benchmark image, dry-runs both pipeline configs, validates output structure |
| <code>.github/workflows/deploy.yml</code>                 | <b>Deploy to GCP</b>          | Builds images and deploys to Cloud Run — see <a href="#">Cloud Deployment</a>                      |
| <code>.github/actions/backend-bootstrap</code>            | <i>(composite)</i>            | Python + Poetry + cache + poetry install                                                           |
| <code>.github/actions/frontend-bootstrap</code>           | <i>(composite)</i>            | Node + npm cache + npm ci                                                                          |

Each top-level workflow has its own status badge in the repository [README.md](#) . Workflows are independent of each other — a regression in one doesn't block the others from running, which makes failures easier to localise on the PR checks page.

### When each pipeline runs

Unlike an earlier version of this page claimed, **not every workflow runs on every push**. Triggers differ per workflow:

| Workflow               | Trigger                                                  |
|------------------------|----------------------------------------------------------|
| Backend Linting        | Every push, every PR, any branch                         |
| Frontend Linting       | Every push, every PR, any branch                         |
| Start Web Service      | Every push, any branch (branches: [""**"])               |
| Validate agent configs | PR opened / synchronize / reopened                       |
| Verify SBOM            | Push to main OR prod, every PR, manual workflow_dispatch |

|                       |                                                                                                                                  |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------------|
| Benchmark Environment | Push to feature/benchmarking-environment, manual workflow_dispatch                                                               |
| Deploy to GCP         | workflow_run — fires after <b>Start Web Service</b> completes successfully on branch <b>prod</b> — plus manual workflow_dispatch |

There are no `paths:` filters on the lint/smoke workflows — every push and PR on any branch runs all three of them, so the team always sees the same, predictable core checks. Inside each workflow, jobs are chained with `needs:` so a broken stage short-circuits the run.

Each workflow has its own `concurrency` group so a fresh push to the same branch cancels the previous in-flight run for that workflow only.

## What the core pipelines do

Each of the three always-on workflows checks out the repo and uses the shared bootstrap composite (Python+Poetry+venv cache, or Node+npm cache) at the start of every job.

```

flowchart LR
    subgraph BL [Backend Linting workflow]
        direction LR
        B1[1 Install] --> B2[2 Lint] --> B3[3 Format] --> B4[4 Tests]
    end
    subgraph FL [Frontend Linting workflow]
        direction LR
        F1[1 Install] --> F2[2 Lint] --> F3[3 Format]
    end
    subgraph WS [Start Web Service workflow]
        direction LR
        W1[Backend boot &#43; /health]
        W2[Frontend production build]
    end

```

### Backend Linting ( backend-lint.yml )

| Job id (for act -j) | Display name            | What it runs                    |
|---------------------|-------------------------|---------------------------------|
| install             | 1. Install              | Bootstrap + import smoke check  |
| lint                | 2. Lint (ruff check)    | Bootstrap + ruff check .        |
| format              | 3. Format (ruff format) | Bootstrap + ruff format --check |
| tests               | 4. Tests (pytest)       | Bootstrap + pytest              |

- Python `3.11` matches `backend/.python-version` and `backend/Dockerfile` .
- `ruff` is configured in `backend/pyproject.toml` ( `[tool.ruff]` ).
- `pytest` configuration lives in `[tool.pytest.ini_options]` . All current tests use mocks for LiveKit/Postgres, so no real services or secrets are required in CI.

### Frontend Linting ( frontend-lint.yml )

| Job id (for act -j) | Display name     | What it runs              |
|---------------------|------------------|---------------------------|
| install             | 1. Install       | Bootstrap + tool versions |
| lint                | 2. Lint (ESLint) | Bootstrap + npm run lint  |

|        |                              |                                  |
|--------|------------------------------|----------------------------------|
| format | 3. Format (Prettier --check) | Bootstrap + npm run format:check |
|--------|------------------------------|----------------------------------|

- Node 20 LTS.
- ESLint flat config in frontend/eslint.config.js , Prettier config in frontend/.prettierrc.json , ignore lists in frontend/.prettierignore .
- The lint step fails on **errors** only. Warnings are printed but do not fail the build. To gate on a specific warning, raise its severity to `error` in the flat config.
- No test step — there is no frontend test suite yet.

**Start Web Service ( start-web-service.yml )**

A smoke-test pipeline that proves both halves of the stack actually *start* — not just lint clean.

| Job id (for act - j) | Display name                | What it runs                                                                                 |
|----------------------|-----------------------------|----------------------------------------------------------------------------------------------|
| backend              | Backend — boot & /health    | Boots taskorbit-api on 127.0.0.1:8000, polls /health, asserts "status":"ok", then tears down |
| frontend             | Frontend — production build | npm run build (tsc -b && vite build); checks dist/index.html exists                          |

- The backend job runs with APP\_ENV=production so uvicorn does **not** enable auto-reload.
- /health does not touch the database, so this smoke test does not require Postgres or any other external service.
- On failure, the API log is uploaded as a workflow artifact ( backend-smoke-api-log ) for offline inspection.

**Validate agent configs ( validate-agent-configs.yml )**

Runs python3 scripts/validate\_agent\_configs.py on every PR (opened/synchronize/reopened) to catch malformed agent configuration JSON before it merges.

**Verify SBOM ( verify-sbom.yml )**

Installs Syft v1.38.2 , runs ./scripts/generate-sbom.sh to regenerate Documentation/sbom.cyclonedx.json and frontend/src/generated/legal-notices.json , then runs git diff --exit-code against both files. If regeneration produces a diff, the committed artifacts are stale and the job fails with a message telling the author to re-run the script and commit. See [SBOM & Legal Notices](#).

**Benchmark Environment ( benchmark.yml )**

Issue #86. Validates the benchmarking harness (build → dry-run both pipeline configs → check results.jsonl / index.csv structure → upload results/ as an artifact) using --dry-run so no provider API keys are needed in CI. Real provider runs happen locally or in scheduled jobs, not in this workflow. See [Component Benchmarking](#).

**Deploy to GCP ( deploy.yml )**

Builds and deploys to Cloud Run after the smoke test passes on prod . This is documented in full in [Cloud Deployment](#) — including the current static-key GCP authentication and the pending Workload Identity migration — rather than duplicated here.

**Caching**

| Layer                 | Cached path   | Key                                                          |
|-----------------------|---------------|--------------------------------------------------------------|
| Backend pip wheels    | ~/.cache/pip  | Managed by actions/setup-python (cache: pip)                 |
| Backend Poetry venv   | backend/.venv | poetry-v1- $\{\{ \text{hashFiles('backend/poetry.lock')} \}$ |
| Frontend npm registry | ~/.npm        | Managed by actions/setup-node (cache: npm)                   |

The `.venv` cache key includes the lockfile hash, so changing `backend/poetry.lock` automatically invalidates the cache. If a cache ever gets poisoned (e.g. a partial install), bump the `v1` segment in the cache key in `.github/actions/backend-bootstrap/action.yml`.

---

## Pre-commit hooks (recommended)

The repository ships a `pre-commit` config at `.pre-commit-config.yaml` that runs the same lint/format checks **automatically on every `git commit`**, scoped to the files you actually staged.

What runs on commit:

- **Generic hygiene** — trim trailing whitespace, fix end-of-file newline, check YAML/TOML, check for merge conflict markers, block large files (>1 MB).
- **Backend** — `ruff --fix` and `ruff format` for staged `.py` files under `backend/`.
- **Frontend** — `eslint --fix` and `prettier --write` for staged `.ts / .tsx / .css` files under `frontend/`, using the project's own `frontend/node_modules/.bin/` binaries.

## One-time setup (Poetry-based, recommended)

`pre-commit` is already declared in `backend/pyproject.toml`'s dev dependencies (`pre-commit = "^4.0.0"`), so `poetry install` in `backend/` is the only install you need.

```
cd backend && poetry install --with dev && cd ..
poetry -C backend run pre-commit install
cd frontend && npm install && cd ..
```

After the second command, every `git commit` automatically runs the hooks against the files you staged. Hooks that auto-fix (ruff, prettier, eslint) write the corrected file to disk; the commit aborts so you can re-stage and re-run `git commit`.

## Alternative: install pre-commit standalone

```
pip install --user pre-commit      # system Python
brew install pre-commit            # macOS Homebrew
pipx install pre-commit            # if you use pipx
```

Then run `pre-commit install` from the repo root.

## Useful commands

```
poetry -C backend run pre-commit run --all-files  # run every hook against the whole repo
poetry -C backend run pre-commit run ruff --all-files # run a single hook
poetry -C backend run pre-commit autoupdate      # bump rev pins in .pre-commit-config.yaml
git commit --no-verify                          # bypass hooks (use sparingly)
```

*Note: bypassing with `--no-verify` is fine for emergency commits, but CI still runs the same checks on the PR, so the failure shifts there.*

---

## Run the same checks locally

### Backend

```
cd backend
poetry install --with dev
poetry run ruff check .
poetry run ruff format --check .
```

```
poetry run pytest

# Auto-fix:
poetry run ruff check --fix .
poetry run ruff format .
```

## Frontend

```
cd frontend
npm ci
npm run lint
npm run format:check

# Auto-fix:
npm run lint -- --fix
npm run format
```

## SBOM

```
./scripts/generate-sbom.sh
git diff --exit-code Documentation/sbom.cyclonedx.json frontend/src/generated/legal-notices.json
```

## Agent config validation

```
python3 scripts/validate_agent_configs.py
```

## Troubleshooting common failures

| Symptom in the failed job                                      | Likely cause                                                                  | Fix                                                                                                  |
|----------------------------------------------------------------|-------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| ruff check reports errors                                      | New code violates a lint rule                                                 | Run <code>poetry run ruff check --fix .</code> then commit.                                          |
| ruff format --check exits non-zero                             | A file in the diff isn't ruff-formatted                                       | Run <code>poetry run ruff format .</code> and commit.                                                |
| pytest fails on a single test                                  | Genuine regression                                                            | Reproduce locally with <code>poetry run pytest tests/path/to/test_file.py::test_name -x -vv</code> . |
| pytest fails on <code>ModuleNotFoundError</code>               | A new dep was added but <code>poetry.lock</code> wasn't committed             | Commit the updated <code>poetry.lock</code> .                                                        |
| eslint complains about <code>Could not find config file</code> | <code>frontend/eslint.config.js</code> deleted or moved                       | Restore the flat config; ESLint v9 requires it.                                                      |
| prettier --check flags many files                              | A new contributor's editor isn't running Prettier                             | Run <code>npm run format</code> once and commit.                                                     |
| <code>npm ci</code> fails with lock file out of sync           | <code>package.json</code> changed without re-running <code>npm install</code> | Run <code>npm install</code> , commit <code>package-lock.json</code> .                               |
| Cached venv looks stale                                        | A force-pushed <code>poetry.lock</code> collided with an older cache key      | Bump the v1 token in <code>.github/actions/backend-bootstrap/action.yml</code> .                     |

|                                                              |                                                                            |                                                                                                                                                                                                |
|--------------------------------------------------------------|----------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Verify SBOM fails on <code>git diff --exit-code</code>       | SBOM/legal-notices artifacts weren't regenerated after a dependency change | Run <code>./scripts/generate-sbom.sh</code> and commit the result.                                                                                                                             |
| Validate agent configs fails                                 | A PR changed agent config JSON in a way that fails schema validation       | Run <code>python3 scripts/validate_agent_configs.py</code> locally and fix the reported config.                                                                                                |
| poetry: command not found after the install step             | <code>~/.local/bin</code> not on PATH                                      | Already handled — the workflow appends it to <code>\$GITHUB_PATH</code> .                                                                                                                      |
| Start Web Service: API never became healthy                  | FastAPI crashed on import or failed to bind <code>:8000</code> within 30s  | Download the <code>backend-smoke-api-log</code> artifact. Reproduce locally with <code>cd backend &amp;&amp; APP_ENV=production poetry run taskorbit-api --host 127.0.0.1 --port 8000</code> . |
| Start Web Service: npm run build fails on a TypeScript error | A new <code>.ts/.tsx</code> change introduces a type error                 | Reproduce locally with <code>cd frontend &amp;&amp; npm run build</code> .                                                                                                                     |
| Deploy to GCP fails to authenticate                          | GCP_SA_KEY secret missing, expired, or rotated                             | Check <b>Settings</b> → <b>Secrets and variables</b> → <b>Actions</b> ; see <a href="#">Cloud Deployment</a> for the current auth model.                                                       |
| Pre-commit hook auto-fixed files and the commit aborted      | Expected — ruff/prettier/eslint rewrite the file so you can review the fix | <code>git add -u</code> and re-run <code>git commit</code> .                                                                                                                                   |

If a step's logs aren't enough to diagnose the issue, click the failing job and download the run logs as an artifact for offline inspection.

## How to extend the pipeline

### Add type checking (recommended next step)

The projects already configure type checkers; they just aren't gated yet.

Backend — add a job between `format` and `tests` in `backend-lint.yml` :

```
typecheck:
  name: "3b. Type-check (mypy)"
  runs-on: ubuntu-latest
  needs: format
  steps:
    - uses: actions/checkout@v4
    - uses: ../.github/actions/backend-bootstrap
      with:
        python-version: ${ env.PYTHON_VERSION }
        poetry-version: ${ env.POETRY_VERSION }
    - name: Type-check with mypy
      working-directory: backend
      run: poetry run mypy src
```

Frontend — add a job after `format` in `frontend-lint.yml` :

```
typecheck:
  name: "4. Type-check (tsc)"
  runs-on: ubuntu-latest
  needs: format
  steps:
```

```

- uses: actions/checkout@v4
- uses: ../github/actions/frontend-bootstrap
  with:
    node-version: ${{ env.NODE_VERSION }}
- name: Type-check with tsc
  working-directory: frontend
  run: npm run type-check

```

Note: `npm run build` in **Start Web Service** already runs `tsc -b`, so a TypeScript regression fails there too. A dedicated `typecheck` job just gives faster feedback.

### Add a frontend test job

```

tests:
  name: "4. Tests"
  runs-on: ubuntu-latest
  needs: format
  steps:
    - uses: actions/checkout@v4
    - uses: ../github/actions/frontend-bootstrap
      with:
        node-version: ${{ env.NODE_VERSION }}
    - name: Run unit tests
      working-directory: frontend
      run: npm test -- --run

```

...and update this document to mention the new step.

### Switch `deploy.yml` to Workload Identity Federation

`deploy.yml` currently authenticates with the static `GCP_SA_KEY` secret. `terraform/modules/iam` already provisions a Workload Identity pool + OIDC provider for this (output `workload_identity_provider`) — the workflow just hasn't been switched over yet. This is the highest-value next change to this pipeline; see [Cloud Deployment](#) for the target auth model.

### Branch protection — merges require CI to pass

The default branch ( `main` ) is protected via GitHub's branch-protection settings (**Settings** → **Branches** → **main**) so that a pull request can be merged only after CI has passed and at least one reviewer has approved. The required status checks are:

- Backend Linting / 4. Tests (pytest)
- Frontend Linting / 3. Format (Prettier --check)
- Start Web Service / Backend — boot & /health
- Start Web Service / Frontend — production build

**Why these four checks?** Each workflow chains its stages with `needs:`, so the terminal job only goes green when every earlier stage did. Requiring just the last job of each chain effectively requires the entire pipeline, and dodges name collisions between the two lint workflows (both have a job called `1. Install`).

`deploy.yml`, `benchmark.yml`, `validate-agent-configs.yml`, and `verify-sbom.yml` are intentionally **not** in this list — they either target `prod` specifically (deploy), run only on a feature branch (benchmark), or gate a narrower concern (config validation, SBOM freshness) than "can this PR merge to main."

### Quick reference

```

# Backend Linting workflow
cd backend

```



```
poetry install --with dev
poetry run ruff check . && poetry run ruff format --check . && poetry run pytest

# Frontend Linting workflow
cd frontend
npm ci
npm run lint && npm run format:check

# Start Web Service workflow
cd backend && APP_ENV=production poetry run taskorbit-api --host 127.0.0.1 --port 8000 &
curl -fsS http://127.0.0.1:8000/health # expect {"status":"ok",...}
kill %1
cd ../frontend && npm run build # tsc + vite build

# Verify SBOM workflow
./scripts/generate-sbom.sh
git diff --exit-code Documentation/sbom.cyclonedx.json frontend/src/generated/legal-notices.json

# Validate agent configs workflow
python3 scripts/validate_agent_configs.py
```

If those commands pass locally, the always-on part of the pipeline will be green. `deploy.yml` and `benchmark.yml` aren't reproducible this way — see [Cloud Deployment](#) and [Component Benchmarking](#) respectively.

---

# Cloud Deployment

## Cloud Deployment Guide

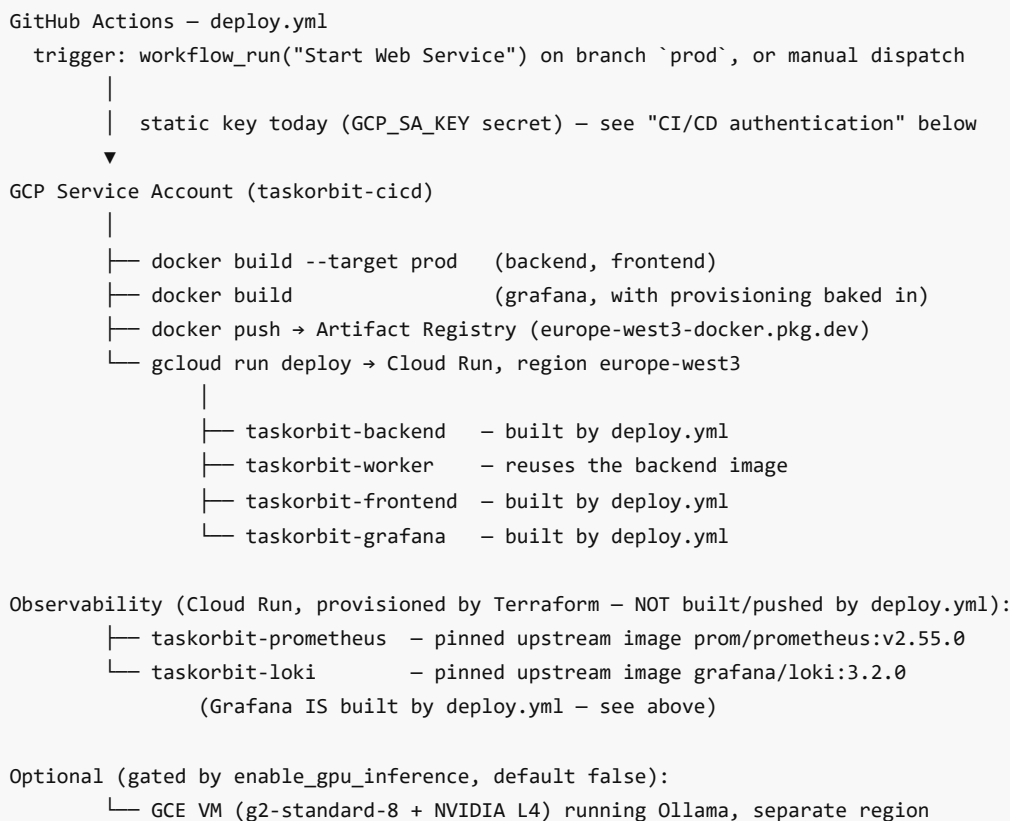
Issue: [#74 Documentation](#)

This page covers how TaskOrbit's `prod` environment works on GCP: how CI/CD authenticates today (and what it's migrating to), how secrets are stored, how automated cost-saving shutdown works, and how developers authenticate locally. See [CI/CD Configuration](#) for the workflow mechanics and [Build & Deploy Documentation](#) for local dev setup.

All GCP resources are in `eu-west-3`, hardcoded in both `deploy.yml` and Terraform's `region` variable default. The optional GPU inference module (self-hosted Ollama) uses a **separate, independently configurable region** (e.g. `us-east1` in `terraform.tfvars.example`) because NVIDIA L4 availability constrains where that VM can go — don't conflate the two.

---

### Architecture overview



`deploy.yml` builds and deploys 4 services. Loki and Prometheus are also Cloud Run services in production, but they run pinned public images provisioned directly by the `observability` Terraform module — they are not part of the CI image build/deploy pipeline described above.

---

### CI/CD authentication

#### Current state: a static service-account key

`deploy.yml` 's own header comment is explicit about this:

```
Authentication uses a GCP service account key stored as a GitHub secret. TODO: switch GCP_SA_KEY to Workload Identity (OIDC) after Step 3 IAM is applied.
```

Both the `build` and `deploy` jobs authenticate via `google-github-actions/auth@v2` using `credentials_json: ${secrets.GCP_SA_KEY}` — a base64-encoded JSON key for the `taskorbit-cicd` service account. This is **not** Workload Identity Federation today, despite the jobs declaring `permissions: id-token: write` (that permission is provisioned for the future OIDC switch and is currently unused).

**What's already provisioned for the migration**

`terraform/modules/iam` **already defines** Workload Identity Federation: `google_iam_workload_identity_pool "github"`, an OIDC provider scoped to this repository via an `attribute.repository` condition, an IAM binding granting the GitHub principal set access to the `taskorbit-cicd` SA, and an output `workload_identity_provider` documented as "set as `GCP_WORKLOAD_IDENTITY_PROVIDER` in GitHub Actions secrets." None of this is wired into `deploy.yml` yet — treat it as infrastructure that's ready for a follow-up PR, not as the current behavior.

**Required GitHub Actions secret (current)**

| Secret name | Purpose                                                                     |
|-------------|-----------------------------------------------------------------------------|
| GCP_SA_KEY  | Base64-encoded JSON key for the <code>taskorbit-cicd</code> service account |

**Required GitHub Actions secrets (post-migration, not yet active)**

| Secret name                    | Where to get the value                                   |
|--------------------------------|----------------------------------------------------------|
| GCP_WORKLOAD_IDENTITY_PROVIDER | terraform output <code>workload_identity_provider</code> |
| GCP_SA_EMAIL                   | The <code>taskorbit-cicd</code> service account email    |

**What the `taskorbit-cicd` service account can do**

This is broader than a minimal CI/CD deploy identity — it's also used for Terraform operations against this project, not just image push/deploy:

| Role                                                                                     | Purpose                                                 |
|------------------------------------------------------------------------------------------|---------------------------------------------------------|
| <code>roles/artifactregistry.writer</code>                                               | Push Docker images                                      |
| <code>roles/run.admin</code>                                                             | Deploy Cloud Run services                               |
| <code>roles/iam.serviceAccountUser</code>                                                | Act as the per-service SAs during deploy                |
| <code>roles/secretmanager.secretAccessor</code> / <code>roles/secretmanager.admin</code> | Read and manage secrets                                 |
| <code>roles/cloudsql.admin</code>                                                        | Manage the Cloud SQL instance                           |
| <code>roles/compute.networkAdmin</code>                                                  | Manage VPC/load-balancer resources                      |
| <code>roles/iam.securityAdmin</code>                                                     | Manage IAM bindings                                     |
| <code>roles/storage.admin</code>                                                         | Manage GCS buckets                                      |
| <code>roles/pubsub.admin</code>                                                          | Manage the Loki log-shipping Pub/Sub topic/subscription |
| <code>roles/logging.admin</code>                                                         | Manage the Cloud Logging sink                           |
| <code>roles/cloudscheduler.admin</code>                                                  | Manage the auto-shutdown scheduler jobs                 |
| <code>roles/serviceusage.serviceUsageAdmin</code>                                        | Enable GCP APIs                                         |

If tightening this to a minimal deploy-only identity (separate from a Terraform-apply identity) is desired, that's a follow-up hardening task, not the current state.

---

## What Terraform provisions

9 modules, applied from `terraform/` :

| Module        | GCP resources                                                                                                                                                                                                                                                     | Key file                                |
|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------|
| network       | VPC, subnet, Cloud Router, NAT gateway, VPC connector                                                                                                                                                                                                             | terraform/modules/network/main.tf       |
| iam           | 5 service accounts (backend, worker, frontend, observability, cicd), Workload Identity pool + provider (unused by CI today), IAM bindings                                                                                                                         | terraform/modules/iam/main.tf           |
| registry      | Artifact Registry repository taskorbit (europe-west3)                                                                                                                                                                                                             | terraform/modules/registry/main.tf      |
| database      | Cloud SQL PostgreSQL, <b>ZONAL availability (single-zone, no regional HA — a deliberate cost tradeoff on shared-core)</b> , point-in-time recovery enabled                                                                                                        | terraform/modules/database/main.tf      |
| secrets       | 18 app secrets prefixed taskorbit- (LLM/STT/TTS/LiveKit/DB/Grafana credentials)                                                                                                                                                                                   | terraform/modules/secrets/main.tf       |
| cloud_run     | backend, worker, frontend Cloud Run v2 services + a Global HTTPS load balancer in front of backend/frontend                                                                                                                                                       | terraform/modules/cloud_run/main.tf     |
| scheduler     | Cloud Scheduler jobs: Cloud SQL stop (22:00 CET) / start (07:00 CET) Mon–Fri                                                                                                                                                                                      | terraform/modules/scheduler/main.tf     |
| observability | loki, prometheus, grafana Cloud Run services, a separate load balancer for Grafana, a GCS log-shipping pipeline (Cloud Logging sink → Pub/Sub → Loki push), plus 3 more secrets (prometheus-config, loki-config, promtail-config)                                 | terraform/modules/observability/main.tf |
| gpu_inference | Optional GCE VM (g2-standard-8 + NVIDIA L4) running Ollama + a GCS bucket for model weights. Gated by <code>enable_gpu_inference</code> (default <code>false</code> — a no-op unless turned on). Uses its own region variable, separate from the main app region. | terraform/modules/gpu_inference/main.tf |

Total secrets across both modules: **21** (18 app secrets + 3 observability config secrets), not the 15 previously documented here — the previous list also omitted OpenRouter entirely. See [Secrets storage and access](#) below for the corrected list.

## Deploying manually (break-glass)

If CI/CD is unavailable and you need to deploy urgently:

```
# Authenticate as yourself
gcloud auth login
gcloud config set project YOUR_PROJECT_ID

# Build and push
docker build --target prod -t europe-west3-docker.pkg.dev/PROJECT/taskorbit/backend:manual ./backend
gcloud auth configure-docker europe-west3-docker.pkg.dev
docker push europe-west3-docker.pkg.dev/PROJECT/taskorbit/backend:manual

# Deploy
gcloud run deploy taskorbit-backend \
  --image europe-west3-docker.pkg.dev/PROJECT/taskorbit/backend:manual \
  --region europe-west3
```

This uses your own `gcloud` identity, not the CI/CD service account — you need at least `roles/artifactregistry.writer` and `roles/run.admin` on the project (or ask an admin to grant them temporarily).

## Secrets storage and access

### Where secrets live

All sensitive values are stored in **GCP Secret Manager** under the prefix `taskorbit-`. 18 are provisioned by `terraform/modules/secrets/`, and 3 more (Prometheus/Loki/Promtail config) by `terraform/modules/observability/`. They are never embedded in container images, environment files, or source code.

| Secret Manager ID                                                                                              | What it holds                                        |
|----------------------------------------------------------------------------------------------------------------|------------------------------------------------------|
| <code>taskorbit-database-url</code>                                                                            | PostgreSQL connection string                         |
| <code>taskorbit-livekit-url</code> / <code>-livekit-api-key</code> / <code>-livekit-api-secret</code>          | LiveKit Cloud credentials                            |
| <code>taskorbit-deepgram-api-key</code> / <code>-deepgram-model</code> / <code>-deepgram-language</code>       | Deepgram (STT or TTS, per agent)                     |
| <code>taskorbit-elevenlabs-api-key</code> / <code>-elevenlabs-voice-id</code> / <code>-elevenlabs-model</code> | ElevenLabs (STT or TTS, per agent)                   |
| <code>taskorbit-openai-api-key</code> / <code>-openai-model</code>                                             | OpenAI provider                                      |
| <code>taskorbit-google-api-key</code> / <code>-google-model</code>                                             | Google Gemini provider                               |
| <code>taskorbit-openrouter-api-key</code> / <code>-openrouter-model</code>                                     | OpenRouter provider                                  |
| <code>taskorbit-grafana-admin-password</code> / <code>-grafana-admin-user</code>                               | Grafana admin login                                  |
| <code>taskorbit-prometheus-config</code>                                                                       | Prometheus scrape config YAML (observability module) |
| <code>taskorbit-loki-config</code>                                                                             | Loki storage config YAML (observability module)      |
| <code>taskorbit-promtail-config</code>                                                                         | Promtail config (observability module)               |

Ollama's `OLLAMA_BASE_URL` / `OLLAMA_MODEL` are passed as **plain env vars**, not secrets — deliberately, to avoid a dependency cycle between the `secrets`, `gpu_inference`, and `iam` modules (see the comment in `main.tf`).

### How Cloud Run services access secrets

Secrets are injected as **environment variables at runtime** via the Cloud Run service definition, not baked into the image. Each Cloud Run service has a service account with `roles/secretmanager.secretAccessor`. Cloud Run fetches the latest secret version on each new instance start — it does **not** hot-reload a running instance when a new version is added.

To update a secret value without redeploying:

```
echo -n "new-value" | gcloud secrets versions add taskorbit-openai-api-key --data-file=-
# The new value takes effect on the next Cloud Run instance start.
# Force a new revision (zero-downtime rolling update):
gcloud run services update taskorbit-backend --region europe-west3 --no-traffic
gcloud run services update-traffic taskorbit-backend --region europe-west3 --to-latest
```

### Rotating secrets

Terraform has `lifecycle { ignore_changes = [secret_data] }` on secret versions so you can rotate keys without Terraform drift. To force Terraform to overwrite a secret (e.g. after a `tfvars` change), taint the version:

```
terraform taint 'module.secrets.google_secret_manager_secret_version.versions["openai-api-key"]'
terraform apply
```

---

## Automatic shutdown for idle cost savings

Cloud Run services scale to zero automatically when there is no traffic — no configuration is needed. The **worker** is typically the exception, running with a nonzero `min_instances` to keep the LiveKit connection warm (configurable via `worker_min_instances`).

Cloud SQL does **not** scale automatically and bills continuously for vCPU and RAM even at `ZONAL` (single-zone, no-HA) tier. Cloud Scheduler jobs stop and start the instance around business hours:

| Job                 | Schedule (CET/CEST) | Action                                                                         |
|---------------------|---------------------|--------------------------------------------------------------------------------|
| taskorbit-sql-stop  | 22:00 Mon–Fri       | Sets Cloud SQL <code>activationPolicy=NEVER</code> (stops billing for compute) |
| taskorbit-sql-start | 07:00 Mon–Fri       | Sets Cloud SQL <code>activationPolicy=ALWAYS</code> (restarts the instance)    |

The jobs call the Cloud SQL Admin REST API directly using an OAuth2 token from the scheduler's service account. No custom code or Cloud Functions are required.

## Checking current Cloud SQL state

```
gcloud sql instances describe taskorbit-postgres --format="value(settings.activationPolicy)"
# ALWAYS = running, NEVER = stopped
```

---

## Local developer authentication

Developers working locally use **Application Default Credentials (ADC)** — no service-account key is required for this path (it's separate from the CI/CD service account discussed above).

### One-time setup

```
gcloud auth login
gcloud auth application-default login
gcloud config set project YOUR_PROJECT_ID
```

## Accessing Secret Manager locally

```
gcloud secrets versions access latest --secret=taskorbit-database-url
gcloud secrets list --filter="name ~ taskorbit"
```

## Connecting to Cloud SQL locally

```
curl -o cloud-sql-proxy https://storage.googleapis.com/cloud-sql-connectors/cloud-sql-proxy/v2.14.2/cloud-sql-proxy.linux.amd64
chmod +x cloud-sql-proxy

./cloud-sql-proxy PROJECT:europe-west3:taskorbit-postgres --port=5432

psql "host=127.0.0.1 port=5432 dbname=taskorbit user=taskorbit"
```

*Note: this is the Cloud SQL proxy port for connecting to the **production** database. It's unrelated to the local `docker-compose` Postgres, which exposes host port **5435** — see [Build & Deploy Documentation](#).*

## Required IAM for local access

| Role                               | Purpose                          |
|------------------------------------|----------------------------------|
| roles/secretmanager.secretAccessor | Read secrets via gcloud / ADC    |
| roles/cloudsql.client              | Connect via Cloud SQL Auth Proxy |
| roles/run.viewer                   | View Cloud Run service status    |

## Production Observability

### Prometheus (metrics scraping)

Prometheus runs as a Cloud Run service ( `taskorbit-prometheus` , pinned image `prom/prometheus:v2.55.0` ) and scrapes both the backend and worker via their public HTTPS Cloud Run URLs.

**Why public URLs instead of internal:** backend and worker both hold `INGRESS_TRAFFIC_ALL` with `allUsers` as `roles/run.invoker` , so Prometheus can reach `/metrics` without a token. Do not add a `credentials_file` or `authorization` block to the Prometheus config — the GKE-only token path does not exist on Cloud Run.

**Config management:** the scrape config is stored in Secret Manager as `taskorbit-prometheus-config` .

```
# 1. Update the secret via Terraform (preferred – keeps state consistent)
terraform apply -target=module.observability

# 2. Redeploy Prometheus to pick up the new secret version
gcloud run services update taskorbit-prometheus \
  --region=europe-west3 --project=amos-taskorbit
```

*Cloud Run fetches secret versions at instance startup — creating a new secret version does **not** automatically reload a running instance. Redeploy to be certain.*

### Loki (log aggregation)

In production there is no Docker daemon — Promtail cannot mount `/var/run/docker.sock` the way it does locally. Instead:

1. Cloud Run stdout → **Cloud Logging** (automatic)
2. A Cloud Logging sink filters `taskorbit-backend` and `taskorbit-worker` log entries
3. The sink publishes to a Pub/Sub topic
4. A Pub/Sub push subscription delivers to Loki's push endpoint with an OIDC token

### Grafana (dashboards)

Grafana runs on Cloud Run behind its own dedicated load balancer (separate from the backend/frontend load balancer). Its image **is** built and pushed by `deploy.yml` (unlike Loki/Prometheus, which run pinned upstream images). Anonymous access is disabled in production — log in with credentials from Secret Manager:

```
gcloud secrets versions access latest --secret=taskorbit-grafana-admin-user
gcloud secrets versions access latest --secret=taskorbit-grafana-admin-password
```

The Grafana image bakes in provisioning files from `infra/grafana/provisioning/` (datasources + dashboard JSON).

### Observability service ingress summary

| Service           | Ingress             | Public IAM       | Why                                                   |
|-------------------|---------------------|------------------|-------------------------------------------------------|
| taskorbit-backend | INGRESS_TRAFFIC_ALL | allUsers invoker | Frontend API calls + Prometheus scrape                |
| taskorbit-worker  | INGRESS_TRAFFIC_ALL | allUsers invoker | Prometheus scrapes <code>/metrics</code> on port 8001 |

|                      |                     |                  |                                          |
|----------------------|---------------------|------------------|------------------------------------------|
| taskorbit-frontend   | INGRESS_TRAFFIC_ALL | allUsers invoker | End-user browser access                  |
| taskorbit-prometheus | INGRESS_TRAFFIC_ALL | allUsers invoker | Grafana queries Prometheus               |
| taskorbit-loki       | INGRESS_TRAFFIC_ALL | allUsers invoker | Pub/Sub push + Grafana queries           |
| taskorbit-grafana    | INGRESS_TRAFFIC_ALL | allUsers invoker | Browser access (login form handles auth) |

All six confirmed against `terraform/modules/cloud_run/main.tf` and `terraform/modules/observability/main.tf` .

---

See also: [CI/CD Configuration](#) · [Build & Deploy Documentation](#) · [System Architecture](#)

---



# Database Setup

## Database Setup

### Overview

TaskOrbit uses PostgreSQL as its database, managed via SQLAlchemy (async) and Alembic for migrations.

### Technology

- **Database:** PostgreSQL 16
- **ORM:** SQLAlchemy (async)
- **Migrations:** Alembic
- **Driver:** psycopg (async)
- **Test Driver:** aioslite (in-memory SQLite for tests)

### Local Development Setup

#### Prerequisites

- Docker Desktop running

#### Start the database

```
docker compose up -d postgres
```

#### Run migrations

```
cd backend
poetry run alembic upgrade head
```

### Conversation Persistence (Auto-create on first message)

#### Behaviour

When `POST /v1/conversations/process` is called, the backend automatically creates a `Conversation` row if one does not exist for the given `conversation_id`. Both the user message and assistant reply are then persisted to `conversation_messages`. This logic lives in `ConversationOrchestrator._persist_messages()` so it benefits both the REST route and the voice path (LiveKit worker).

#### Voice Path

The LiveKit worker opens a per-call `AsyncSession` and passes it to `build_default_agent(db=session, conversation_id=conversation_id)`. The `conversation_id` is generated client-side by the frontend and forwarded through the LiveKit JWT metadata so voice calls persist under the same id the History page uses to load messages.

#### Frontend Wiring

- On page load, `ConversationalChat.tsx` calls `GET /v1/conversations` and displays a count of previous conversations.
- The History page ( `HistoryPage.tsx` ) calls `GET /v1/conversations` to list past conversations and `GET /v1/conversations/{id}/messages` to display messages for a selected conversation.

#### SBOM Impact

No new dependencies added in this feature.

### API Endpoints

Conversations

| Method | Endpoint                                     | Description                                                             |
|--------|----------------------------------------------|-------------------------------------------------------------------------|
| POST   | /v1/conversations                            | Create a new conversation with a generated UUID                         |
| GET    | /v1/conversations                            | Get all conversations (used by frontend on page load)                   |
| GET    | /v1/conversations/{conversation_id}/messages | Get all messages for a conversation                                     |
| POST   | /v1/conversations/process                    | Process a message, auto-create conversation if needed, persist messages |

Example — Get all conversations

```
curl http://localhost:8000/v1/conversations
```

Response:

```
{"conversations": [{"id": "...", "agent_name": "TaskOrbit", "started_at": "..."}]}
```

Schema

users

| Column          | Type     | Description           |
|-----------------|----------|-----------------------|
| id              | Integer  | Primary key           |
| username        | String   | Unique username       |
| email           | String   | Unique email          |
| hashed_password | String   | Hashed password       |
| is_active       | Boolean  | Account status        |
| created_at      | DateTime | Creation timestamp    |
| updated_at      | DateTime | Last update timestamp |

conversations

| Column     | Type     | Description                                                            |
|------------|----------|------------------------------------------------------------------------|
| id         | String   | Primary key (UUID, generated client-side or by POST /v1/conversations) |
| agent_id   | String   | Agent identifier (from request agent_config)                           |
| agent_name | String   | Agent display name (from request agent_config)                         |
| started_at | DateTime | Conversation start                                                     |
| ended_at   | DateTime | Conversation end (nullable)                                            |

conversation\_messages (canonical message store)

| Column | Type    | Description |
|--------|---------|-------------|
| id     | Integer | Primary key |

|                 |          |                                 |
|-----------------|----------|---------------------------------|
| conversation_id | String   | Foreign key to conversations    |
| user_id         | Integer  | Foreign key to users (nullable) |
| role            | String   | Message role (user/assistant)   |
| content         | String   | Message content                 |
| created_at      | DateTime | Message timestamp               |

#### tool\_executions

| Column          | Type     | Description                      |
|-----------------|----------|----------------------------------|
| id              | Integer  | Primary key                      |
| conversation_id | String   | Foreign key to conversations     |
| tool_id         | String   | Tool identifier                  |
| tool_type       | String   | Type of tool                     |
| confirmed       | Boolean  | Whether execution was confirmed  |
| executed_at     | DateTime | Execution timestamp              |
| result          | JSON     | Tool execution result (nullable) |

### Canonical Message Store

`conversation_messages` is the single source of truth for chat messages. It links to both conversations and users. The `chat_histories` table has been removed and replaced by extending `conversation_messages` with a `user_id` column.

### Environment Variables

Copy `.env.example` to `.env` and fill in your credentials:

```
cp .env.example .env
```

### Running Tests

```
cd backend
poetry run pytest tests/test_crud.py -v
poetry run pytest tests/test_conversations_route.py -v
```

Expected output: 11 passed (crud), 9 passed (conversations route)

---

# LiveKit Cloud Setup

## LiveKit Cloud Setup

Developer setup for the LiveKit Cloud integration delivered in [#26](#).

### What you need

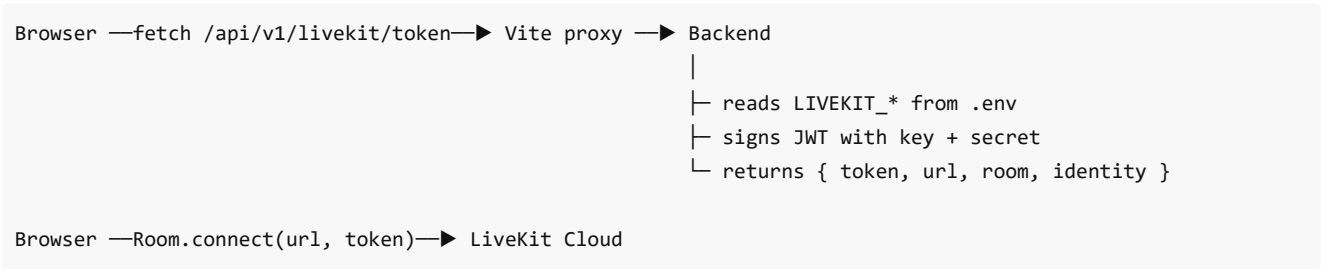
| Variable           | Where it lives                                           | Where to get it                                                                  |
|--------------------|----------------------------------------------------------|----------------------------------------------------------------------------------|
| LIVEKIT_URL        | backend/.env + frontend/.env.local (as VITE_LIVEKIT_URL) | LiveKit Cloud → Settings → Project — looks like wss://your-project.livekit.cloud |
| LIVEKIT_API_KEY    | backend/.env only                                        | LiveKit Cloud → Settings → API keys → <b>Create key</b>                          |
| LIVEKIT_API_SECRET | backend/.env only                                        | Shown once at key creation — copy immediately                                    |

The frontend never sees the API key or secret.

### Steps

1. **Sign in** at [cloud.livekit.io](https://cloud.livekit.io) and pick (or create) your project.
2. **Create an API key** — Settings → API keys → Create key. Copy both the key and the secret to a temporary note before closing the modal.
3. **Find the WebSocket URL** — Settings → Project. Format: `wss://<handle>.livekit.cloud`.
4. **Configure backend** — copy `backend/.env.example` to `backend/.env` and fill in `LIVEKIT_URL`, `LIVEKIT_API_KEY`, `LIVEKIT_API_SECRET`.
5. **Configure frontend** — set `VITE_LIVEKIT_URL` in `frontend/.env.local` to the same URL.
6. **Run** — `docker compose up --build` from the repo root.

### How the token flow works



The backend mints **short-lived JWTs (6h TTL)** scoped to a single `(room, identity)` pair. The frontend uses the JWT directly with `livekit-client` — it never touches the API secret.

### Where the code lives

| File                                        | Role                                          |
|---------------------------------------------|-----------------------------------------------|
| backend/src/taskorbit/api/routes/livekit.py | The POST <code>/v1/livekit/token</code> route |
| backend/src/taskorbit/types.py              | Request / response Pydantic models            |
| backend/src/taskorbit/config.py             | Reads <code>LIVEKIT_*</code> from environment |
| backend/tests/test_livekit_route.py         | Route unit tests                              |

|                                  |                               |
|----------------------------------|-------------------------------|
| frontend/src/lib/livekitToken.ts | Frontend's typed token client |
|----------------------------------|-------------------------------|

---

## Architecture context

This is the *infrastructure* slice — it terminates at "browser holds a valid token and can join a LiveKit room." Audio publish/subscribe lives in [#14](#) (STT) and [#15](#) (TTS); the Python agent worker that joins rooms server-side is owned by a downstream ticket.

See also: [System Architecture](#).

---

## References

- Ticket [#26 — Setup LiveKit Infrastructure](#)
  - LiveKit Cloud docs: [docs.livekit.io](https://docs.livekit.io)
-

# LLM Integration

## LLM Integration

Developer setup for the LLM provider integration delivered in [#18](#).

TaskOrbit supports **two LLM providers out of the box** — OpenAI and Google Gemini. Provider selection is **per-task**, not global: each agent task definition picks its own provider and model via `AgentConfig.llm`. This means a single deployment can run different tasks on different providers.

### What you need

| Variable       | Where it lives    | Where to get it                                                                                  |
|----------------|-------------------|--------------------------------------------------------------------------------------------------|
| OPENAI_API_KEY | backend/.env only | <a href="https://platform.openai.com/api-keys">platform.openai.com/api-keys</a>                  |
| OPENAI_MODEL   | backend/.env only | Default <code>gpt-4o-mini</code> . Used only when an agent config does not specify a model.      |
| GOOGLE_API_KEY | backend/.env only | <a href="https://aistudio.google.com/apikey">aistudio.google.com/apikey</a>                      |
| GOOGLE_MODEL   | backend/.env only | Default <code>gemini-2.5-flash</code> . Used only when an agent config does not specify a model. |

You only need a key for the providers you actually use. If an agent config requests a provider whose key is unset, the factory raises `LLMConfigError` at request time.

The frontend never sees any LLM API key.

### Steps

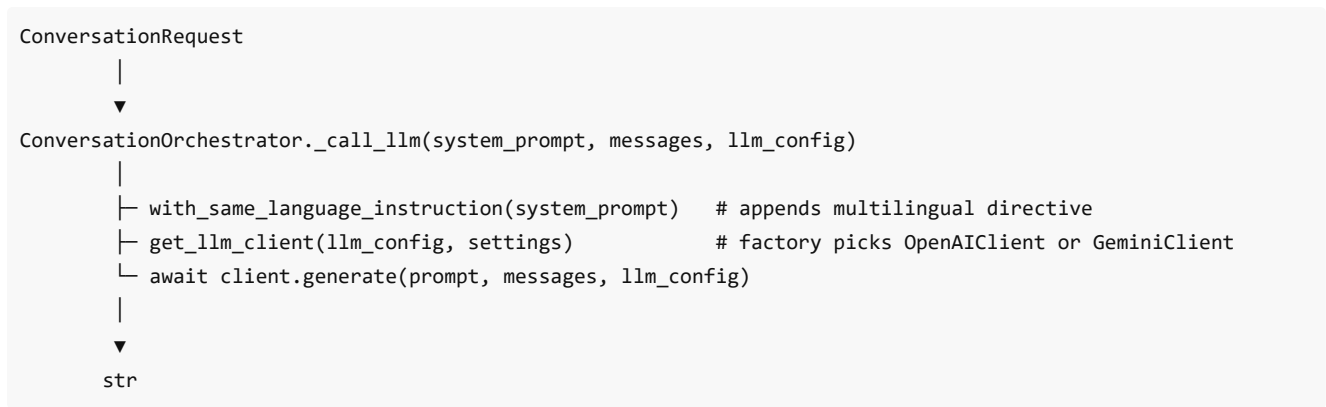
1. **Get the keys** from the dashboards above.
2. **Configure backend** — copy `backend/.env.example` to `backend/.env` and fill in `OPENAI_API_KEY` and/or `GOOGLE_API_KEY`.
3. **Restart backend** — if running in Docker, use `docker compose up -d --force-recreate backend`. A plain `restart` does **not** re-read `env_file`.
4. **Verify** — see the smoke-test section below.

### How per-task routing works

Each agent task definition carries its own `llm` block:

```
// frontend/src/lib/mockAgents.ts
export const JOHN_DOE_AGENT: AgentConfig = {
  name: "John Doe",
  persona: "...",
  llm: {
    provider: "openai",    // or "google"
    model: "gpt-4.1-mini", // any model the provider supports
  },
  // ...
};
```

When the frontend sends a `ConversationRequest`, it includes the full `agent_config`. The orchestrator's `_call_llm` reads `agent_config.llm` and dispatches to the matching concrete client via the factory:



The factory is the single switch point. Adding a third provider means writing one new client and adding one branch to the factory — no orchestration changes.

## Error types

All LLM errors derive from `LLMError`. The hierarchy lets callers handle classes of failure without depending on SDK-specific exception types:

| Error                          | When it's raised                                                                 |
|--------------------------------|----------------------------------------------------------------------------------|
| <code>LLMConfigError</code>    | Provider key missing, model field empty, or unsupported provider in agent config |
| <code>LLMAuthError</code>      | Provider rejects the API key (401/403)                                           |
| <code>LLMRateLimitError</code> | Provider returns rate-limit response (429)                                       |
| <code>LLMTimeoutError</code>   | Provider times out (408 on Google, <code>APITimeoutError</code> on OpenAI)       |
| <code>LLMAPIError</code>       | Any other provider failure, including empty/blocked responses                    |

`LLMAuthError`, `LLMRateLimitError`, and `LLMTimeoutError` all inherit from `LLMAPIError`, so a single `except LLMAPIError` block catches all provider-side failures while still letting callers handle specific cases first.

## Multilingual behavior

`with_same_language_instruction()` (in `prompts.py`) appends a directive to every system prompt instructing the LLM to **respond in the same language the user wrote in**. This is applied uniformly across providers in `_call_llm`, so no per-client work is needed.

## Smoke test

After setting keys and restarting the backend, you can exercise `_call_llm` directly:

```
docker compose exec -T backend python -c "  
import asyncio  
from taskorbit.orchestration import ConversationOrchestrator  
from taskorbit.types import LLMConfig, LLMPProvider, Message, MessageRole  
  
async def main():  
    orch = ConversationOrchestrator()  
    result = await orch._call_llm(  
        'You are a helpful assistant.',  
        [Message(role=MessageRole.USER, content='What is 2+2?')],  
        LLMConfig(provider=LLMPProvider.OPENAI, model='gpt-4o-mini'),  
    )
```

```
print('RESULT:', result)

asyncio.run(main())
"
```

Swap `LLMProvider.OPENAI` / `gpt-4o-mini` for `LLMProvider.GOOGLE` / `gemini-2.5-flash` to smoke Gemini.

### Where the code lives

| File                                                    | Role                                                                                |
|---------------------------------------------------------|-------------------------------------------------------------------------------------|
| backend/src/taskorbit/integrations/llm/base.py          | LLMClient Protocol — the contract both clients satisfy                              |
| backend/src/taskorbit/integrations/llm/factory.py       | <code>get_llm_client(llm_config, settings)</code> — picks the right concrete client |
| backend/src/taskorbit/integrations/llm/errors.py        | Typed error hierarchy                                                               |
| backend/src/taskorbit/integrations/llm/prompts.py       | <code>with_same_language_instruction()</code> helper                                |
| backend/src/taskorbit/integrations/llm/messages.py      | Message → SDK wire-format translation                                               |
| backend/src/taskorbit/integrations/llm/openai_client.py | OpenAI concrete client (uses <code>openai.AsyncOpenAI</code> )                      |
| backend/src/taskorbit/integrations/llm/gemini_client.py | Gemini concrete client (uses <code>google.genai</code> )                            |
| backend/src/taskorbit/orchestration/__init__.py         | <code>_call_llm</code> — the single call site that wires it all up                  |
| backend/src/taskorbit/config.py                         | Reads <code>OPENAI_*</code> and <code>GOOGLE_*</code> from environment              |
| backend/tests/test_llm_factory.py                       | Factory tests (provider routing + config validation)                                |
| backend/tests/test_llm_prompts.py                       | Same-language helper tests                                                          |
| backend/tests/test_messages_helper.py                   | Message-translation tests                                                           |
| backend/tests/test_openai_client.py                     | OpenAI client unit tests (mocked SDK)                                               |
| backend/tests/test_gemini_client.py                     | Gemini client unit tests (mocked SDK)                                               |
| backend/tests/test_call_llm_wiring.py                   | Integration test for <code>_call_llm</code> wiring                                  |

### Architecture context

This is the **provider-integration slice** — it terminates at "given a system prompt + message history + LLM config, return assistant text." It does not handle tool calls, streaming, or memory across sessions; those are downstream tickets.

The wrapper layer is deliberately thin: SDK imports are confined to `openai_client.py` and `gemini_client.py`, so swapping or adding providers never touches orchestration code. See [System Architecture](#) for the broader runtime picture.

### References

- Ticket [#18 — Basic LLM Integration](#)
- OpenAI Python SDK: [github.com/openai/openai-python](https://github.com/openai/openai-python)
- Google GenAI Python SDK: [googleapis.github.io/python-genai](https://googleapis.github.io/python-genai)



# Self-Hosted Inference

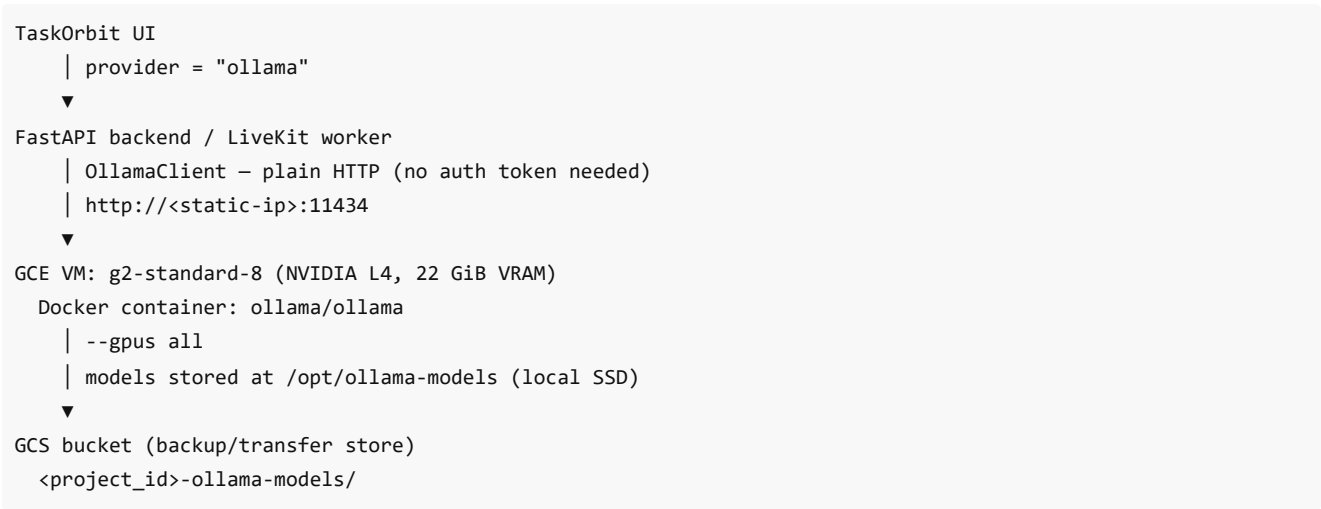
## Self-Hosted Inference (Ollama on GCE VM with NVIDIA L4)

TaskOrbit can run Gemma and Qwen models on a dedicated **NVIDIA L4 GPU** via an [Ollama](#) container on a Google Compute Engine VM. This keeps inference costs predictable, keeps data fully within your GCP project, and eliminates external API rate limits.

Self-hosted inference runs alongside the existing cloud providers (OpenAI, Gemini, OpenRouter) — you select it per-agent in the UI.

**Why GCE VM instead of Cloud Run?** Cloud Run GPU nodes are locked to NVIDIA driver 535. Ollama 0.30+ requires driver 550+. GCE Deep Learning VM images ship with driver 580 and CUDA 12.9 pre-installed, enabling current model families (gemma4, qwen3).

### Architecture



- **Auth:** None — the GCE VM uses a static external IP on port 11434 with a firewall rule scoped to `taskorbit-ollama` tagged instances. `OllamaClient._needs_auth = False` ; no GCP identity token is fetched.
- **Models:** Stored on the VM's local SSD at `/opt/ollama-models` (mounted into the container at `/root/.ollama` ). Local SSD gives fast random-read access for tensor loading (~10–15 s cold start vs 3+ minutes from GCSFuse).
- **GCS bucket:** Used as a persistent backup and transfer store — not mounted at runtime. Copy weights to/from the VM manually with `gsutil` or `docker exec` .
- **Thinking mode disabled:** `OllamaClient` sends `"think": false` at the top level of every `/api/chat` request. This suppresses extended thinking for Qwen3 and gemma4 model families so `content` is always populated.
- **Multi-tenancy:** The Ollama container runs with `OLLAMA_NUM_PARALLEL=4` (concurrent requests per model), `OLLAMA_MAX_LOADED_MODELS=4` (models kept in VRAM simultaneously), `OLLAMA_MAX_QUEUE=100` (request backpressure queue), and `OLLAMA_FLASH_ATTENTION=1` (optimised attention kernel). Physical VRAM (22 GiB) is the hard limit — see [VRAM Constraints](#) below.
- **Startup model pre-pull:** All models in `var.gpu_inference_models` are pulled to local SSD automatically on VM boot, avoiding slow first-request downloads.

### Supported Models

| Model                    | VRAM   | Context | Notes                                     |
|--------------------------|--------|---------|-------------------------------------------|
| <code>gemma4:e4b</code>  | ~6 GB  | 128K    | Lightest; fastest response                |
| <code>gemma4:26b</code>  | ~18 GB | 128K    | <b>Default</b> — best speed/quality on L4 |
| <code>qwen3.5:9b</code>  | ~7 GB  | 32K     | Compact Qwen; good for intent routing     |
| <code>qwen3.6:27b</code> | ~17 GB | 256K    | Best for long-context tasks               |

All four models are pre-pulled to local SSD at VM boot by the startup script. Up to `OLLAMA_MAX_LOADED_MODELS=4` can be resident in VRAM simultaneously, but physical VRAM (22 GiB) limits which combinations fit — see [VRAM Constraints](#). The active model for each agent is set via `OLLAMA_MODEL` in backend `.env` or per-agent in the UI.

---

## Prerequisites

- GCP project with NVIDIA L4 quota in the target zone ( `us-east1-b` by default)
- `gcloud` CLI authenticated: `gcloud auth login`
- Terraform  $\geq 1.7$ , `enable_gpu_inference = true` in `terraform.tfvars`

Request L4 GPU quota at: **GCP Console** → **IAM & Admin** → **Quotas** — search for `NVIDIA_L4_GPUS` in `us-east1` .

---

## 1 — Enable (First-Time Deploy)

### Step 1a — Configure Terraform

```
# terraform/terraform.tfvars
enable_gpu_inference = true
gpu_inference_region = "us-east1"
gpu_inference_zone   = "us-east1-b"
ollama_version       = "latest"
ollama_model         = "gemma4:26b"
gpu_inference_models = ["gemma4:e4b", "gemma4:26b", "qwen3.6:27b", "qwen3.5:9b"]
```

### Step 1b — Apply

```
cd terraform
terraform apply -target=module.gpu_inference
```

This provisions the static IP, firewall rule, and GCE VM. The VM startup script:

1. Waits for the NVIDIA driver (pre-installed on Deep Learning VM images)
2. Installs Docker and `nvidia-container-toolkit` if not already present
3. Runs `nvidia-ctk runtime configure --runtime=docker --set-as-default` unconditionally — this is required even on Deep Learning VM images which ship with the toolkit pre-installed; without `--set-as-default` , `--gpus all` silently falls back to CPU
4. Starts the Ollama container with multi-tenancy environment variables
5. Pre-pulls all models in `gpu_inference_models` to local SSD

### Step 1c — Note the static IP

```
terraform output -module=gpu_inference ollama_service_url
# e.g. http://35.231.129.211:11434
```

Set this as `OLLAMA_BASE_URL` in `backend/.env` for local dev, or update the secret in Secret Manager for Cloud Run.

### Step 1d — Verify models were pre-pulled

The startup script automatically pulls all models in `gpu_inference_models` to local SSD. Verify:

```
# From your local machine
curl http://<static-ip>:11434/api/tags | jq '.models[].name'
```

If a model is missing (e.g. the VM was provisioned before the pre-pull loop was added), pull it manually:

```
gcloud compute ssh taskorbit-ollama-inference --zone=us-east1-b
docker exec ollama ollama pull gemma4:26b
```

### Step 1e — Validate

```
# From your local machine
curl http://<static-ip>:11434/api/tags | jq '.models[].name'

# Or run the full validation script
./backend/scripts/validate_ollama_inference.sh
```

## VRAM Constraints

The NVIDIA L4 has **22 GiB VRAM**. `OLLAMA_MAX_LOADED_MODELS=4` is a config-level cap; physical VRAM is the real limit.

| Combination              | VRAM required | Fits in 22 GiB?                                                     |
|--------------------------|---------------|---------------------------------------------------------------------|
| gemma4:e4b + qwen3.5:9b  | ~13 GB        | Yes — both stay loaded                                              |
| gemma4:26b alone         | ~18 GB        | Yes                                                                 |
| qwen3.6:27b alone        | ~17 GB        | Yes                                                                 |
| gemma4:26b + qwen3.6:27b | ~35 GB        | No — switching evicts the previous model (~10–15 s reload from SSD) |
| All 4 models             | ~48 GB        | No                                                                  |

To keep all large models simultaneously loaded without eviction, upgrade to an **NVIDIA A100 80 GB** ( `a2-highgpu-1g` machine type).

## 2 — Start and Stop

### Start the VM

```
gcloud compute instances start taskorbit-ollama-inference \
  --zone=us-east1-b --project=amos-taskorbit
```

The Ollama Docker container starts automatically via the `--restart unless-stopped` policy. The first inference request after a cold boot takes ~10–15 s for model loading.

### Stop the VM (save costs when idle)

```
gcloud compute instances stop taskorbit-ollama-inference \
  --zone=us-east1-b --project=amos-taskorbit
```

GCE billing stops immediately when the VM is stopped. Models on the local SSD are preserved across stop/start cycles (the disk is not deleted).

**Cost:** A *g2-standard-8* with *NVIDIA L4* costs approximately \$1.30/hour. Stopping overnight (14 h) and on weekends saves ~60% vs 24/7 operation.

## 3 — Swap the Active Model

Update `OLLAMA_MODEL` in `backend/.env` (local) or Secret Manager (Cloud Run):

```
# Local dev
echo "OLLAMA_MODEL=qwen3.6:27b" >> backend/.env

# Cloud Run (production)
echo -n "qwen3.6:27b" | gcloud secrets versions add ollama-model \
  --data-file=- --project=amos-taskorbit
```

The new model takes effect on the next backend restart. No Terraform change needed.

---

## 4 — Add a New Model

### Pull directly on the VM

```
gcloud compute ssh taskorbit-ollama-inference --zone=us-east1-b
docker exec ollama ollama pull gemma4:31b
```

### Backup to GCS (optional)

```
# On the VM – copy model weights to the GCS bucket for safe-keeping
gsutil -m cp -r /opt/ollama-models/models/gemma4 \
  gs://amos-taskorbit-ollama-models/
```

### Add to the UI dropdown

Open `frontend/src/lib/pipelineOptions.ts` and append the tag to the `ollama` array:

```
ollama: [
  "gemma4:26b",
  "gemma4:e4b",
  "qwen3.6:27b",
  "qwen3.5:9b",
  "gemma4:31b",    // ← add here
],
```

No backend change needed.

---

## 5 — Remove a Model

```
gcloud compute ssh taskorbit-ollama-inference --zone=us-east1-b
docker exec ollama ollama rm gemma4:e4b
```

To also free up GCS backup storage:

```
gsutil rm -r gs://amos-taskorbit-ollama-models/gemma4/
```

---

## 6 — Update Ollama to a New Version

Pin the version in `terraform.tfvars` :

```
ollama_version = "0.7.0"
```

Then re-apply. Terraform recreates the VM instance, which re-runs the startup script with the new Ollama version. **Models on the local SSD are lost** — you must re-pull them after the new VM starts.

To preserve models across version upgrades, back them up to GCS first:

```
gsutil -m cp -r /opt/ollama-models/ gs://amos-taskorbit-ollama-models/
```

## 7 — Selecting Ollama in the UI

1. Open the **Agent Configuration** page
2. Go to the **Pipeline** tab
3. Set **LLM Provider** → `ollama (self-hosted)`
4. Choose a model from the **Model** dropdown ( `gemma4:26b` is the default)
5. Save the agent

The agent now routes all LLM calls through the self-hosted GCE VM.

## 8 — Validate a Running Instance

```
VM_IP="35.231.129.211" # replace with your static IP

# List loaded models
curl http://$VM_IP:11434/api/tags | jq '.models[].name'

# Test a completion (native API, think=false)
curl -X POST http://$VM_IP:11434/api/chat \
  -H "Content-Type: application/json" \
  -d '{"model":"gemma4:26b","think":false,"stream":false,"messages":[{"role":"user","content":"Say hello"}]}'
```

To verify requests are reaching the self-hosted endpoint via the backend, check logs:

```
gcloud logging read \
  'jsonPayload.event="llm_client_selected" AND jsonPayload.provider="ollama"' \
  --project=amos-taskorbit --limit=5 --format=json
```

Or in Grafana (Loki):

```
{job="cloud-run-logs"} |= "llm_client_selected" | json | provider = "ollama"
```

## Cost Reference

| Scenario                        | Hourly cost | Weekly cost (estimate) |
|---------------------------------|-------------|------------------------|
| g2-standard-8 + NVIDIA L4, 24/7 | ~\$1.30/h   | ~\$218                 |
| Work hours only (Mon–Fri 08–22) | ~\$1.30/h   | ~\$91                  |
| VM stopped (idle)               | \$0         | \$0                    |

GCS model storage: ~\$0.02/GB/month (negligible for 48 GB of model weights).

## Troubleshooting

**Ollama computing on CPU / using system RAM instead of VRAM** Deep Learning VM images ship with `nvidia-container-toolkit` pre-installed, so the install block in the startup script is skipped — but the NVIDIA runtime still needs to be wired into Docker. SSH into the VM and run:

```
sudo nvidia-ctk runtime configure --runtime=docker --set-as-default
sudo systemctl restart docker
sudo docker rm -f ollama
# Then recreate the container (copy the docker run command from the startup script)
```

Verify GPU access inside the container:

```
sudo docker exec ollama nvidia-smi
```

The startup script already runs `nvidia-ctk runtime configure --runtime=docker --set-as-default` unconditionally, so newly provisioned VMs are not affected.

**Port 11434 unreachable from backend** Check the firewall rule exists and the VM has the `taskorbit-ollama` network tag:

```
gcloud compute firewall-rules describe taskorbit-ollama-api --project=amos-taskorbit
gcloud compute instances describe taskorbit-ollama-inference \
  --zone=us-east1-b --format="value(tags.items)"
```

**/api/tags returns empty model list** Models have not been pulled yet. SSH into the VM and run `docker exec ollama ollama pull gemma4:26b`.

**Inference request times out** The `OllamaClient` uses a 300 s timeout. If you see consistent timeouts on the first request after a restart, the model is still loading into VRAM (~10–15 s). Retry once.

**content field is empty, reasoning contains text** The model is in thinking mode. Verify `OllamaClient` is sending `"think": false` at the top level of the `/api/chat` request body (not inside `options`). The OpenAI-compatible `/v1/chat/completions` endpoint does **not** forward this flag — always use `/api/chat`.

**intent\_router\_parse\_error in backend logs** The intent router could not extract valid JSON from the LLM response. This typically means the model returned preamble text before the JSON object. The router uses `re.search(r"\{[^\}]*\}", raw)` to extract the first JSON block — if the response format is unusual, check the raw LLM output in logs.

**OLLAMA\_BASE\_URL not set error** Set `OLLAMA_BASE_URL=http://<static-ip>:11434` in `backend/.env` (local) or in the `ollama-base-url` Secret Manager secret (Cloud Run).

**VM startup script did not complete** Check serial console output:

```
gcloud compute instances get-serial-port-output taskorbit-ollama-inference \
  --zone=us-east1-b --project=amos-taskorbit | tail -50
```

---

# SBOM & Legal Notices

Issue: [#138 SBOM Automation & Legal Notices UI](#)

## User story

As a developer  
I want the application to automatically generate an SBOM and display the third-party licenses as Legal Notices in the UI  
So that I can view the open-source licenses and legal credits of the libraries used in the application

## Overview

TaskOrbit uses [Syft](#) to scan all application dependencies and publish them in two places:

1. **Machine-readable SBOM** — `Documentation/sbom.cyclonedx.json` (CycloneDX JSON 1.5)
2. **In-app Legal Notices** — searchable dialog in the application footer

Every open-source package in the backend and frontend is inventoried. Licenses are displayed automatically — SPDX IDs where available, `UNKNOWN` where metadata is missing (blind accept: nothing is hidden).

## Implementation status

| Area                                                                  | Status                        |
|-----------------------------------------------------------------------|-------------------------------|
| Syft SBOM generation ( <code>scripts/generate-sbom.sh</code> )        | ✓ Done                        |
| CycloneDX artifact ( <code>Documentation/sbom.cyclonedx.json</code> ) | ✓ Done — 247 components       |
| License extraction ( <code>scripts/extract_legal_notices.py</code> )  | ✓ Done                        |
| Legal Notices UI (footer dialog)                                      | ✓ Done — 245 components shown |
| Unit tests ( <code>test_extract_legal_notices.py</code> )             | ✓ Done — 8/8 passing          |
| CI verification ( <code>.github/workflows/verify-sbom.yml</code> )    | ✓ Done                        |
| Sprint planning BOM ( <code>Deliverables/sprint-12/</code> )          | ✓ Done                        |

## How it works

### Developer workflow

```
# Prerequisites: brew install syft
./scripts/generate-sbom.sh
```

The script:

1. Scans `backend/` with Syft (Poetry packages)
2. Scans `frontend/` with Syft (npm packages)
3. Merges both into `Documentation/sbom.cyclonedx.json`
4. Extracts `frontend/src/generated/legal-notices.json` for the UI

Run after any change to `backend/poetry.lock` or `frontend/package-lock.json`, then commit both generated files with the lockfile change.

### In the application

The frontend bundles `legal-notices.json` at build time. Users click **Legal Notices** in the footer to search packages, filter by license, and view component counts.

### CI/CD integration

SBOM verification is part of the project's CI pipeline.

| Workflow        | Triggers                    | What it does                                                          |
|-----------------|-----------------------------|-----------------------------------------------------------------------|
| verify-sbom.yml | Every push + every PR       | Regenerates SBOM artifacts and confirms they match what is committed  |
| deploy.yml      | After quality gates on prod | Frontend image includes the committed <code>legal-notices.json</code> |

This keeps the SBOM in sync with lockfiles automatically — if a developer updates dependencies without regenerating, CI catches it before merge.

### Scope

| Layer                     | Included                                                  |
|---------------------------|-----------------------------------------------------------|
| Backend (Python / Poetry) | ✓                                                         |
| Frontend (npm / React)    | ✓                                                         |
| Terraform providers       | Documented separately in <code>terraform/README.md</code> |

### Artifacts

| File                                                        | Purpose                      |
|-------------------------------------------------------------|------------------------------|
| <code>Documentation/sbom.cyclonedx.json</code>              | Full CycloneDX JSON 1.5 SBOM |
| <code>frontend/src/generated/legal-notices.json</code>      | License list for the UI      |
| <code>Deliverables/sprint-12/planning-documents.xlsx</code> | Sprint Bill of Materials     |

#### Current scan:

- **SBOM components:** 247
- **Legal notices (UI):** 245
- **Last generated:** `2026-07-02T06:42:43Z`

### Verification

```
./scripts/generate-sbom.sh
cd backend && poetry run pytest tests/test_extract_legal_notices.py -v
cd frontend && npm run dev # → footer "Legal Notices"
```

### Acceptance criteria

| # | Criterion                              | Status                                    |
|---|----------------------------------------|-------------------------------------------|
| 1 | SCA tool generates up-to-date SBOM     | ✓ Syft → <code>sbom.cyclonedx.json</code> |
| 2 | Dependencies scanned with blind accept | ✓ All listed; unknown → UNKNOWN           |



|   |                                        |                               |
|---|----------------------------------------|-------------------------------|
| 3 | Planning Documents SBOM updated        | ✔ Sprint 12 Bill of Materials |
| 4 | Legal Notices section/modal in UI      | ✔ Footer link + dialog        |
| 5 | UI renders auto-generated license info | ✔ Imports legal-notices.json  |

---

**Related**

- [Documentation/sbom.md](#) — operational guide
  - [Documentation/sbom-testing-guide.md](#) — testing checklist
  - [Syft documentation](#)
- 

**Changelog**

| Date       | Change                                                                     |
|------------|----------------------------------------------------------------------------|
| 2026-07-02 | Wiki page published; Sprint 12 BOM updated; end-to-end automation verified |
| 2026-07-01 | SBOM generation + Legal Notices UI shipped (#138)                          |